What's new

# A tool to verify estimates, II: a flexible proof assistant

Terence Tao · 9 May 2025

https://terrytao.wordpress.com/2025/05/09/a-tool-to-verify-estimates-ii-a-flexible-proof-assistant/

tao2typ · converted to Typst 2026-10-14 (UTC)

---

In a recent post, I talked about a proof of concept tool to verify estimates automatically. Since that post, I have overhauled the tool twice: first to turn it into a rudimentary proof assistant that could also handle some propositional logic; and second into a much more flexible proof assistant (deliberately designed to mimic the Lean proof assistant in several key aspects) that is also powered by the extensive Python package sympy for symbolic algebra, following the feedback from previous commenters. This I think is now a stable framework with which one can extend the tool much further; my initial aim was just to automate (or semi-automate) the proving of asymptotic estimates involving scalar functions, but in principle one could keep adding tactics, new sympy types, and lemmas to the tool to handle a very broad range of other mathematical tasks as well.

The current version of the proof assistant can be found here. (As with my previous coding, I ended up relying heavily on large language model assistance to understand some of the finer points of Python and sympy, with the autocomplete feature of Github Copilot being particularly useful.) While the tool can support fully automated proofs, I have decided to focus more for now on semi-automated interactive proofs, where the human user supplies high-level "tactics" that the proof assistant then performs the necessary calculations for, until the proof is completed.

It's easiest to explain how the proof assistant works with examples. Right now I have implemented the assistant to work inside the interactive mode of Python, in which one enters Python commands one at a time. (Readers from my generation may be familiar with text adventure games, which have a broadly similar interface.) I would be interested developing at some point a graphical user interface for the tool, but for prototype purposes, the Python interactive version suffices. (One can also run the proof assistant within a Python script, of course.)

After downloading the relevant files, one can launch the proof assistant inside Python by typing `from main import *` and then loading one of the pre-made exercises. Here is one such exercise:

```
>>> from main import *
>>> p = linarith_exercise()
Starting proof.  Current proof state:
x: pos_real
y: pos_real
z: pos_real
h1: x < 2*y
```

```
h2: y < 3*z + 1
|- x < 7*z + 2
```

This is the proof assistant's formalization of the following problem: If $x, y, z$ are positive reals such that $x < 2y$ and $y < 3z + 1$, prove that $x < 7z + 2$.

The way the proof assistant works is that one directs the assistant to use various "tactics" to simplify the problem until it is solved. In this case, the problem can be solved by linear arithmetic, as formalized by the `Linarith()` tactic:

```
>>> p.use(Linarith())
Goal solved by linear arithmetic!
Proof complete!
```

If instead one wanted a bit more detail on how the linear arithmetic worked, one could have run this tactic instead with a `verbose` flag:

```
>>> p.use(Linarith(verbose=true))
Checking feasibility of the following inequalities:
1*z > 0
1*x + -7*z >= 2
1*y + -3*z < 1
1*y > 0
1*x > 0
1*x + -2*y < 0
Infeasible by summing the following:
1*z > 0 multiplied by 1/4
1*x + -7*z >= 2 multiplied by 1/4
1*y + -3*z < 1 multiplied by -1/2
1*x + -2*y < 0 multiplied by -1/4
Goal solved by linear arithmetic!
Proof complete!
```

Sometimes, the proof involves case splitting, and then the final proof has the structure of a tree. Here is one example, where the task is to show that the hypotheses $(x > -1) \wedge (x < 1)$ and $(y > -2) \wedge (y < 2)$ imply $(x + y > -3) \wedge (x + y < 3)$:

```
>>> from main import *
>>> p = split_exercise()
Starting proof.  Current proof state:
x: real
y: real
h1: (x > -1) & (x < 1)
h2: (y > -2) & (y < 2)
|- (x + y > -3) & (x + y < 3)
>>> p.use(SplitHyp("h1"))
Decomposing h1: (x > -1) & (x < 1) into components x > -1, x < 1.
1 goal remaining.
>>> p.use(SplitHyp("h2"))
Decomposing h2: (y > -2) & (y < 2) into components y > -2, y < 2.
1 goal remaining.
```

```
>>> p.use(SplitGoal())
Split into conjunctions: x + y > -3, x + y < 3
2 goals remaining.
>>> p.use(Linarith())
Goal solved by linear arithmetic!
1 goal remaining.
>>> p.use(Linarith())
Goal solved by linear arithmetic!
Proof complete!
>>> print(p.proof())
example (x: real) (y: real) (h1: (x > -1) & (x < 1)) (h2: (y > -2) & (y < 2)): (x
+ y > -3) & (x + y < 3) := by
  split_hyp h1
  split_hyp h2
  split_goal
  . linarith
  linarith
```

Here at the end we gave a "pseudo-Lean" description of the proof in terms of the three tactics used: a tactic `cases h1` to case split on the hypothesis `h1`, followed by two applications of the `simp_all` tactic to simplify in each of the two cases.

The tool supports asymptotic estimation. I found a way to implement the order of magnitude formalism from the previous post within sympy. It turns out that sympy, in some sense, already natively implements nonstandard analysis: its symbolic variables have an `is_number` flag which basically corresponds to the concept of a "standard" number in nonstandard analysis. For instance, the `sympy` version `S(3)` of the number 3 has `S(3).is_number == True` and so is standard, whereas an integer variable `n = Symbol("n", integer=true)` has `n.is_number == False` and so is nonstandard. Within `sympy`, I was able to construct orders of magnitude `Theta(X)` of various (positive) expressions `X`, with the property that `Theta(n)=Theta(1)` if `n` is a standard number, and use this concept to then define asymptotic estimates such as $X \lesssim Y$ (implemented as `lesssim(X,Y)`). One can then apply a logarithmic form of linear arithmetic to then automatically verify some asymptotic estimates. Here is a simple example, in which one is given a positive integer $N$ and positive reals $x, y$ such that $x \leq 2N^2$ and $y < 3N$, and the task is to conclude that $xy \lesssim N^4$:

```
>>> p = loglinarith_exercise()
Starting proof.  Current proof state:
N: pos_int
x: pos_real
y: pos_real
h1: x <= 2*N**2
h2: y < 3*N
|- Theta(x)*Theta(y) <= Theta(N)**4
>>> p.use(LogLinarith(verbose=True))
Checking feasibility of the following inequalities:
Theta(N)**1 >= Theta(1)
Theta(x)**1 * Theta(N)**-2 <= Theta(1)
Theta(y)**1 * Theta(N)**-1 <= Theta(1)
Theta(x)**1 * Theta(y)**1 * Theta(N)**-4 > Theta(1)
Infeasible by multiplying the following:
```

```
Theta(N)**1 >= Theta(1) raised to power 1
Theta(x)**1 * Theta(N)**-2 <= Theta(1) raised to power -1
Theta(y)**1 * Theta(N)**-1 <= Theta(1) raised to power -1
Theta(x)**1 * Theta(y)**1 * Theta(N)**-4 > Theta(1) raised to power 1
Proof complete!
```

The logarithmic linear programming solver can also handle lower order terms, by a rather brute force branching method:

```
>>> p = loglinarith_hard_exercise()
Starting proof.  Current proof state:
N: pos_int
x: pos_real
y: pos_real
h1: x <= 2*N**2 + 1
h2: y < 3*N + 4
|- Theta(x)*Theta(y) <= Theta(N)**3
>>> p.use(LogLinarith())
Goal solved by log-linear arithmetic!
Proof complete!
```

I plan to start developing tools for estimating function space norms of symbolic functions, for instance creating tactics to deploy lemmas such as Holder's inequality and the Sobolev embedding inequality. It looks like the `sympy` framework is flexible enough to allow for creating further object classes for these sorts of objects. (Right now, I only have one proof-of-concept lemma to illustrate the framework, the arithmetic mean-geometric mean lemma.)

I am satisfied enough with the basic framework of this proof assistant that I would be open to further suggestions or contributions of new features, for instance by introducing new data types, lemmas, and tactics, or by contributing example problems that ought to be easily solvable by such an assistant, but are currently beyond its ability, for instance due to the lack of appropriate tactics and lemmas.

## Comments (20)



**David Roberts** · 9 May 2025

Typo "$(y>-2) wedge (y<2)$" (unprocessed LaTeX)

*[Corrected, thanks – T.]*



**Anonymous** · 11 May 2025

minor typo: In this case, the problem can be solved by linear arithmetic, as formalize dby the `Linarith()` tactic:

*[Corrected, thanks – T.]*



**Anonymous** · 11 May 2025

Is there a particular reason why you decided to use SymPy/Python for this project, and not Lean4?

 **Terence Tao** · 12 May 2025

I do eventually plan to have the ability to output the proof into a Lean certificate, but it will be significantly faster to proceed initially using the (not rigorously verified) Python/sympy tools rather than the fully verified Lean code, as the task of formally verifying proofs in Lean is still an order of magnitude slower than that of generating pseudo-proofs in say Python.

 **Guanyuming He** · 13 May 2025

Dear Prof. Tao,

If I understand correctly, your tool is like a collection of very high-level (w.r.t. proof assistants) tactics that are not fully verified down to basic axioms, because that would take too long. Are you interested in extending the tool to handle other less analysis-related jobs which are also tedious to do in a fully verified proof assistant?

More abstractly, I think an analogy can be made about your tool and compilers: many compilers can be divided into the front-end and the back-end, where the front-end analyzes the source code (making sure it is syntactically correct and semantically sound) and generates some intermediate representation of the source code, and where the back-end constructs the desired program from the intermediate presentation.

Do you like the idea to generalize your tool to act as some kind of front-end that transforms human proof ideas into some intermediate representation which are not immediately verifiable by proof assistants? This division makes such a process more modular and decouples the tool-chain, having many benefits. I don't know if other people already have explored such an idea; your tool reminds me of the front/back-end division of compilers.

Thank you,
Guanyuming He

 **Anonymous** · 11 May 2025

This is a very interesting tool, the asymptotic estimation examole is great!

Hopefully it quickly gets a largish crowd of contributors. This might be accelarated if some form of credit sytem is put in place (like a bimontly announcement of recent changes and valuable contributors, or MO like badges).

 **brieflyhideout36ac0bd093** · 11 May 2025

This tool can be used to prove asymptotic estimators, but is there a tool that can help to derive them? After all, to prove something, you need to have the object of proof in the first place.

 **Terence Tao** · 12 May 2025

I initially view the use case for this tool as a way to certify proofs that were initially generated by traditional pen-and-paper calculations, without having to go through the more lengthy process of directly generating a proof in a full-featured proof assistant such as Lean. Once this more lightweight tool becomes stable and one has some moderately large data set of existing proofs using this tool, I could imagine that machine learning

tools could then be applied to explore the proof space more, and in particular to discover new estimates that are variants of those in the training data, and supply proof certificates for them (perhaps through a two-step process in which a lightweight proof assistant such as this one is used to first produce a pseudocode proof, which is then formalized in Lean in a semi-automated or automated fashion using some other tool).

 **Aidan Backus** · 12 May 2025

A family of inequalities which might make good “targets” is given by Chapter 10 of Lindqvist’s book on the p-Laplacian (https://lqvist.folk.ntnu.no/p-laplace.pdf). These inequalities mostly follow from the usual suspects of fundamental theorem of calculus, Cauchy–Schwarz, convexity of x mapsto |x|^p, etc., but the manipulations involved are not so obvious and one may need more involved tactics to be able to prove them.

 **Terence Tao** · 12 May 2025

Thanks for the suggestion! I’ll have to think about how to fit these sorts of manipulations into the current framework.

 **Anonymous** · 12 May 2025

Will you consider giving estimated probabilities (i.e. suggestions) for the next move to make in a proof? Or is that more advanced than intended?

 **Terence Tao** · 12 May 2025

I’m hoping that that sort of functionality can eventually be supplied by a separate layer of tools, probably powered by some sort of machine learning, once one has enough of a corpus of proofs performed using this sort of assistant to serve as suitable training data.

 **philzook58** · 13 May 2025

I’m interested to see where this goes! I’ve been working on a related tool in the sense that it is an interactive proof assistant in python based on z3 and to a lesser degree sympy. https://github.com/philzook58/knuckledraggerhttps://github.com/philzook58/knuckledragger I have wondered what it would look like to switch that emphasis but found sympy to do an uncomfortable amount of unstoppable simplification under my feet, and to be developed from the CAS tradition more than a logical tradition, lacking good support for things like quantifiers, sorts, sets, or systematic logical discipline. Still very feature rich and closer to the desires, needs, and aesthetics of some engineers, physicists, and mathematicians. Asymptotic estimates was not on my radar as a good challenge problem but looks fun!

 **Terence Tao** · 13 May 2025

Yes, I’m finding sympy’s simplifier to be a little too aggressive for this application. I used it in some of my tactics but I have since removed it as it was causing unwanted behavior. I may add it back as an optional flag though.

 **Christopher** · 21 May 2025

Somewhat tangential, but are you familiar with internal set theory?

It is an approach to nonstandard analysis. I remember in a previous post you mentioned how in principle, you could iterate nonstandard analysis to give multiple “levels” of infinity, such that the next level sees the previous level as finite. IST natively supports this (you don’t need

multiple levels of IST, it already does recursive nonstandard analysis). Not that iterating the levels manually is too hard, but it saves you some work.

It is an axiomatic approach which is *conservative* over ZFC. In particular, it adds a "is standard" predicate such that st(0), st(1), st(2), st(3), ... are all provable but so is exists H. forall n. st(n)⟹n < H. This isn't the novel part, of course. It can also prove much more than this.

Proof that H exists: by axiom I there is a finite set G that contains every standard set, (since this is set theory, everything is sets). Let H be the maximum natural number in G. (Notice this proves that G isn't *just* the standard sets. Any set containing all standard natural numbers must overspill. Also, by transfer, st(n) implies st(n+1).) Likewise, we can get an infinitesimal either as 1/H, or by taking the smallest positive real number in G.

Next I'll show two of the ***novel*** things IST can do:

1. Given a parameter p (such as an infinite natural number, or a nonstandard sequence), define a nonstandard natural number that exceeds everything *definable* from p. (Where by definable we always mean in the language of set theory *without* st.)
2. Given an infinite natural number H, define a natural number J of a *lower* level of infinity. That is, J is infinite but can not be used to define anything larger than H. H is infinite relative to J.

And I'll combine them as well.

One) Let G be a finite set such for every standard f, we have f(p) ∈ G. This exists by axiom I. Since every definable set in standard (by axiom of transfer, the definition must have a standard solution, and since it is a definition it is the unique solution), we have that the maximum natural number in G bounds every natural number definable from p.

Two) Let's generalize a bit. Let B be a set such that every element of B is nonstandard. In our case, B is the set of natural numbers greater than or equal to H. Let G be a finite set such that for all standard x, we have x ∈ G, and if x is a function, we have forall y ∈ G. x(y) ∉ B. This exists by axiom I (since all the elements of B are nonstandard). Then the maximum natural number J in G exceeds every standard natural number, but every natural number definable from J is not in B, and so is less than H.

Combination) Let B be a set that does not contain the image of p under any standard function. (For example, the natural numbers greater than or equal to the H in the first theorem.) Let G be a finite set such that for every standard function f we have f(p) ∈ G and forall y ∈ G. f(p, y) ∉ B. Than the maximum natural number J in G exceeds anything definable from p but cannot define anything in B, even with help from p.

Note that you can replace "maximum natural number" with "maximum element in a directed set" in any of these examples. Just additionally stipulate that G has a maximum when using axiom I.

>  **Christopher** · 6 June 2025
>
> Oh, even tho IST is interesting I just realized that these two properties aren't too hard to achieve. In the semantic approach to nonstandard analysis, all you need is for the model to be countably saturated, or even just computably saturated. Consider a model M of, say, ZFC.

1) Finding a natural number H that is bigger than anything definable from parameters p (and thus 1/H will be an infinitesimal smaller than anything definable from p). Create a type with, for each formula φ(p, n) in the language of ZFC, the formula Ψ(H, p) := [∃ n ∈ ℕ. φ(p, n) => ∃ n ≤ H. φ(p, n)]. This is finitely satisfiable (given a finite list of Ψ, just let H be greater than a solution from each), and so by saturation there is a H that satisfies all of them. In particular, H will be greater than any natural number definable from p.

2) Finding a natural number K that is bigger than anything definable from parameters p, but such that H (satisfying the conclusion of the previous block) is bigger than anything definable from (p, K). Create a type with, for each formula φ_1(p, n) in the language of ZFC, the formula Ψ_1(K, p, H) := [∃ n ∈ ℕ. φ_1(p, n) => ∃ n ≤ K. φ_1(p, n)], and also for each formula φ_2(p, K, n) in the language of ZFC, the formula Ψ_2(K, p, H) := [∃ n ∈ ℕ. φ_2(p, K, n) => ∃ n ≤ H. φ_2(p, K, n)]. This again is finitely satisfiable. Given a finite list of Ψ, there exists N and K such that K bounds the solutions for n in the finitely many φ_1(p, n) and N bounds the solutions for n in the finitely many φ_2(p, K, n). By our hypothesis about H, we can find such a N that is ≤ H. Thus we can find K satisfying the type.

**Anonymous** · 12 July 2025

What is the PA based on? Is it type theory, classical set theory, or ...?



**Terence Tao** · 13 July 2025

The PA is not designed for complete formal soundness; it uses tools such as sympy that are not certified in any formal frameworks, but are instead coded in Python and so may contain bugs like any other conventional software that has not been formally verified. I would imagine though that once a proof has been obtained in this "leaky" proof assistant, it should be relatively straightforward to translate this proof to a fully formalized proof (in one's favorite foundations of mathematics, such as type theory or set theory), in a semi-automated or fully automated fashion.

**Anonymous** · 16 July 2025

Dear Prof. Tao,
I'd like to work on this from the CS side, for I feel a very elegant solution is present.

Currently I'm looking for a better understanding of the problem space, i.e. the intended use cases of this tool. (for, currently class Basic contains many parameters that is never used in the examples)

Relatedly, I wish to understand if a 'numerical test' is possible (fitting larger numbers into variables) for your intended use case, which may provide insight to whether a direction of symbolic reasoning is correct.

Cheers! I'm quite impressed by the project, especially its well-documentedness.

**-** · 14 October 2025

We are happy to announce our tool O-Forge, which can prove the questions mentioned in this blogpost, as well as those mentioned in Professor Tao's earlier MathOverflow post. You can try it here- https://o-forge.com. The GitHub repo is here- https://github.com/ayushkhaitanrutgers/decompositions-2

We are very grateful to Professor Tao for suggesting several improvements to our tool and website, which have certainly made them much better.

We will also be grateful for any feedback from the users.

Some context: I was the OP on the MO post, and I spent quite some time trying to modify Alphageometry for this purpose. However, we eventually realized this summer that the set of algebraic manipulations was many orders of magnitude larger than that of plane geometry constructions, and hence this approach might not be a perfect fit here. It was only then that we learned about Mathematica's Resolve[] function, which can rigorously complete proofs using first order logic.

We have integrated Mathematica with a frontier LLM to first divide a domain into the "correct" subdomains, which will facilitate an easy proof of the estimate, and then prompt pass code to Mathematica through an API call to complete the proof. Thanks very much, and we look forward to hearing from you. My email ID is ayush.khaitan@rutgers.edu